

RAPPORT TECHNIQUE

Module Security Center

Architecture, Mecanismes et Outils d'Analyse de Securite

Projet	Automatisation Intelligente des Tests Applicatifs
Module	Security Center (SAST / DAST / SCA)
Stack	Spring Boot 4 + Next.js 16 + PostgreSQL 15
Outils	Semgrep, OWASP ZAP, OWASP Dependency-Check
Date	Juin 2026

Table des Matieres

1. Introduction
2. Architecture Technique
3. Outils d'Analyse de Securite
4. Modele de Donnees
5. API REST
6. Frontend - Security Center Dashboard
7. Rapport PDF de Securite
8. Prerequis d'Installation
9. Diagramme de Sequence - Scan SAST
10. Diagramme de Sequence - Scan DAST
11. Classification des Vulnerabilites
12. Conclusion

1. Introduction

Le module Security Center integre trois types d'analyse de securite (SAST, DAST, SCA) directement dans la plateforme d'automatisation des tests. Il permet de scanner le code source, les applications web en production, et les dependances d'un projet Java/Spring Boot.

Ce module transforme la plateforme d'un simple outil d'automatisation de tests en une solution complete de DevSecOps, offrant une couverture de securite a 360 degres. Les resultats sont centralises dans PostgreSQL, visualises dans un dashboard interactif Next.js, et exportables en rapport PDF professionnel.

- **SAST (Static Application Security Testing)** - Analyse du code source via Semgrep
- **DAST (Dynamic Application Security Testing)** - Scan d'applications en production via OWASP ZAP
- **SCA (Software Composition Analysis)** - Audit des dependances via OWASP Dependency-Check
- **Dashboard temps reel** - KPIs, OWASP Top 10, score de securite, tendances
- **Rapports PDF** - Generation automatique via iTextPDF 7
- **Gestion des vulnerabilites** - Suivi, assignation, classification CWE/OWASP

2. Architecture Technique

2.1 Architecture Globale

```
Frontend (Next.js, port 3000)
| POST /api/security/scan/{sast|dast|sca}
v
ms-execution (Spring Boot, port 8083)
+-- SecurityScanController (REST API)
+-- SecurityScanService (Orchestrateur)
| +-- SemgrepScanService (SAST)
| +-- ZapScanService (DAST)
| +-- DependencyCheckService (SCA)
+-- SecurityReportService (PDF iTextPDF)
+-- JPA Repositories --> PostgreSQL

ms_gestion (Spring Boot, port 8082)
+-- SecurityController (Dashboard/KPIs)
+-- SecurityService (Lecture/Aggregation)
+-- SecurityDataSeeder (Donnees demo)

PostgreSQL (port 5432)
+-- security_scans
+-- security_vulnerabilities
+-- compliance_results
```

2.2 Flux d'execution d'un scan

1. L'utilisateur selectionne un projet et un environnement dans le dashboard
2. Le frontend envoie POST /api/security/scan/{type}?projectId=X&environmentId=Y
3. ms-execution cree un enregistrement SecurityScan (status=RUNNING)
4. Le scan s'execute en asynchrone (@Async Spring)
5. Pour SAST/SCA : clone Git du repo (JGit + PAT auth) puis execution de l'outil et parsing JSON
6. Pour DAST : Docker lance OWASP ZAP contre l'URL cible de l'environnement
7. Les resultats sont persistes dans la table security_vulnerabilities
8. Le scan passe en COMPLETED avec duree et metriques
9. Le rapport PDF est disponible via GET /api/security/report/{scanId}

3. Outils d'Analyse de Securite

3.1 SAST - Semgrep

Semgrep est un outil d'analyse statique open-source qui detecte les vulnerabilites de securite dans le code source. Il supporte plus de 30 langages et dispose de 2500+ regles communautaires couvrant les principales failles de securite.

Commande d'execution :

```
semgrep scan --config auto --json --output report.json <repo>
```

Detections principales :

CWE	Vulnerabilite	OWASP Top 10
CWE-89	SQL Injection	A03 - Injection
CWE-79	Cross-Site Scripting (XSS)	A03 - Injection
CWE-78	OS Command Injection	A03 - Injection
CWE-798	Hardcoded Credentials	A07 - Auth Failures
CWE-327	Broken Cryptography	A02 - Crypto Failures
CWE-22	Path Traversal	A01 - Broken Access Control
CWE-502	Unsafe Deserialization	A08 - Integrity Failures
CWE-918	SSRF	A10 - SSRF

- **Sortie parsee** : fichier, ligne, snippet de code, CWE ID, severite, recommandation
- **Mapping OWASP** : chaque CWE est mappe vers la categorie OWASP Top 10 (A01-A10)
- **Classe Java** : SemgrepScanService.java - parse le JSON et cree les SecurityVulnerability

3.2 DAST - OWASP ZAP

OWASP ZAP (Zed Attack Proxy) est l'outil de reference pour l'analyse dynamique des applications web. Il est execute via Docker en mode headless (baseline scan) pour detecter les vulnerabilites sur une application en cours d'execution.

Commande d'execution :

```
docker run --rm --network host zapproxy/zap-stable zap-baseline.py -t <url> -J report.json -I
```

Detections :

- XSS reflechi et stocke
- CSRF manquant
- Headers de securite absents (CSP, HSTS, X-Frame-Options)
- Cookies non securises (HttpOnly, Secure, SameSite)
- CORS misconfiguration
- Information disclosure
- **Prerequis** : Docker installe, application cible accessible via baseUrlApi ou baseUrlWeb
- **Classe Java** : ZapScanService.java - lance le conteneur Docker et parse le JSON
- **Timeout** : configurable via security.zap.timeout-minutes (default: 15 min)

3.3 SCA - OWASP Dependency-Check

OWASP Dependency-Check analyse les dependances Maven (pom.xml) pour identifier les composants avec des CVE (Common Vulnerabilities and Exposures) connues dans la base NVD (National Vulnerability

Database).

Commande d'execution :

```
mvn dependency-check:check -DfailBuildOnCVSS=11 -Dformat=JSON
```

- **Base de donnees** : NVD - mise a jour automatique a chaque execution
- **Detections** : Log4Shell (CVE-2021-44228), Spring4Shell (CVE-2022-22965), Jackson RCE, etc.
- **Injection automatique** : le plugin Maven dependency-check-maven 9.0.9 est injecte dans le pom.xml si absent
- **Classe Java** : DependencyCheckService.java - injecte le plugin, execute Maven, parse le JSON

4. Modele de Donnees

4.1 Entite SecurityScan

Champ	Type	Description
id	BIGSERIAL	Cle primaire auto-generee
scanRef	VARCHAR	Identifiant unique (SCAN-XXXXXXXX)
scanType	ENUM	SAST, DAST, SCA
engine	VARCHAR	Semgrep, OWASP ZAP, Dependency-Check
status	ENUM	RUNNING, COMPLETED, FAILED, SCHEDULED
startedAt	TIMESTAMP	Debut du scan
completedAt	TIMESTAMP	Fin du scan
duration	VARCHAR	Duree formatee (ex: 2m 34s)
linesAnalyzed	INTEGER	Lignes de code analysees (SAST)
filesAnalyzed	INTEGER	Fichiers analyses (SAST)
coverage	DOUBLE	Couverture estimee
branch	VARCHAR	Branche Git scannee
commitHash	VARCHAR	Hash du commit
targetUrl	VARCHAR	URL cible (DAST)
project_id	BIGINT FK	Reference vers le projet

4.2 Entite SecurityVulnerability

Champ	Type	Description
id	BIGSERIAL	Cle primaire
title	VARCHAR	Titre de la vulnerabilite
severity	ENUM	CRITICAL, HIGH, MEDIUM, LOW, INFO
vulnType	ENUM	SAST, DAST, SCA
source	VARCHAR	Outil source
status	ENUM	OPEN, IN_PROGRESS, RESOLVED, CLOSED, FALSE_POSITIVE
cweld	VARCHAR	Identifiant CWE (ex: CWE-89)
owaspCategory	VARCHAR	Categorie OWASP Top 10 (A01-A10)
file	VARCHAR	Fichier source (SAST)
line	INTEGER	Numero de ligne (SAST)
snippet	TEXT	Extrait de code vulnerable
endpoint	VARCHAR	Endpoint HTTP (DAST)
httpMethod	VARCHAR	Methode HTTP (DAST)

Champ	Type	Description
parameter	VARCHAR	Parametre vulnerable (DAST)
description	TEXT	Description detaillee
risk	TEXT	Evaluation du risque
recommendation	TEXT	Recommandation de correction
fixExample	TEXT	Exemple de code corrige
evidence	TEXT	Preuve d'exploitation (DAST)
scan_id	BIGINT FK	Reference vers le scan
project_id	BIGINT FK	Reference vers le projet

5. API REST

5.1 Endpoints ms-execution (port 8083) - Execution des scans

Methode	Endpoint	Description
POST	/api/security/scan/sast	Lance un scan SAST (Semgrep)
POST	/api/security/scan/dast	Lance un scan DAST (OWASP ZAP)
POST	/api/security/scan/sca	Lance un scan SCA (Dependency-Check)
GET	/api/security/scans?projectId=	Liste les scans d'un projet
GET	/api/security/scans/{id}	Detail d'un scan
GET	/api/security/scans/{scanId}/vulns	Vulnerabilites d'un scan
GET	/api/security/report/{scanId}	Telecharger le rapport PDF
GET	/api/security/report/project/{id}	Rapport PDF consolide

5.2 Endpoints ms_gestion (port 8082) - Dashboard et KPIs

Methode	Endpoint	Description
GET	/api/security/dashboard	KPIs agreges (score, compteurs, OWASP)
GET	/api/security/vulnerabilities	Liste toutes les vulnerabilites
PATCH	/api/security/vulnerabilities/{id}/status	Modifier le statut
PATCH	/api/security/vulnerabilities/{id}/assign	Assigner a un developpeur
GET	/api/security/compliance	Resultats de conformite

6. Frontend - Security Center Dashboard

6.1 Architecture des composants

Composant	Description
OverviewCards	6 KPI cards (Total, Critical, High, Medium, Low, Resolved)
SecurityScoreGauge	Score circulaire SVG avec sous-scores
SastCenter	Dashboard SAST avec metriques, historique, findings expandables
DastCenter	Dashboard DAST avec findings par endpoint, evidence, actions
OwaspTable	Tableau OWASP Top 10 avec score de risque par categorie
VulnerabilityTable	Table de donnees avec recherche, filtres, pagination
ComplianceCards	4 frameworks (OWASP Top 10, ASVS, ISO 27001, NIST CSF)
SecurityTrends	Graphiques Recharts (area, donut, bar chart)
PipelineView	Visualisation CI/CD avec 8 stages et gate de securite
ReportSection	Telechargement de rapports (PDF/CSV/JSON)

6.2 Integration Backend - Graceful Fallback

Le frontend utilise un mecanisme de fallback transparent : il tente d'abord de charger les donnees depuis le backend (ms_gestion). Si le backend est indisponible, il utilise les donnees statiques de demonstration. Un badge indique la source des donnees.

1. useEffect() appelle loadFromBackend() au montage du composant
2. fetchDashboard() et fetchVulnerabilities() sont appeles en parallele (Promise.all)
3. Si les reponses sont valides : mise a jour du state avec les donnees live
4. Si erreur ou backend down : le state conserve les donnees statiques initiales
5. Badge affiche : 'Live - ms_gestion' (vert) ou 'Static Data' (jaune)

7. Rapport PDF de Securite

Le rapport PDF est genere par SecurityReportService (ms-execution) utilisant iTextPDF 7. Il est disponible par scan individuel ou consolide par projet.

Section	Contenu
Page de couverture	Identifiant du scan, moteur, statut, niveau de risque global
Resume executif	Tableau KPI (Total/Critical/High/Medium/Low/Info) + barre de severite
Configuration	Moteur, duree, lignes/fichiers analyses, branche, commit
Tableau des vulns	Vue tabulaire de toutes les findings avec severite et CWE
Findings detailles	Cards pour chaque vuln Critical/High avec snippet et recommandation
Recommandations	Priorisation par severite + distribution OWASP Top 10

8. Prerequis d'Installation

Composant	Installation	Version
Semgrep	pip install semgrep	>= 1.0
Docker	docker.com/get-docker	>= 20.x
OWASP ZAP	docker pull zaproxy/zap-stable	Latest
Maven	Système ou wrapper (mvnw)	>= 3.8
PostgreSQL	Docker ou natif	15
Java	JDK 17+	17

9. Diagramme de Sequence - Scan SAST

```
User --> Frontend : Cliquer 'Lancer SAST'
Frontend --> ms-execution : POST /api/security/scan/sast?projectId=1&environmentId;=1
ms-execution --> DB : INSERT security_scans (status=RUNNING)
ms-execution --> Frontend : 202 Accepted {scanRef: 'SCAN-A1B2C3D4'}
ms-execution --> GitHub : git clone (JGit + PAT authentication)
GitHub --> ms-execution : Repository clone
ms-execution --> Semgrep : semgrep scan --config auto --json
Semgrep --> ms-execution : JSON results (findings[])
ms-execution --> Parse : findings[] --> SecurityVulnerability[]
ms-execution --> DB : INSERT security_vulnerabilities (N lignes)
ms-execution --> DB : UPDATE security_scans (status=COMPLETED)
--- Telechargement du rapport ---
User --> Frontend : Cliquer 'Telecharger Rapport'
Frontend --> ms-execution : GET /api/security/report/{scanId}
ms-execution --> DB : SELECT scan + vulnerabilities
ms-execution --> iTextPDF : Generation PDF
ms-execution --> Frontend : PDF binary
Frontend --> User : Telechargement du fichier
```

10. Diagramme de Sequence - Scan DAST

```
User --> Frontend : Cliquer 'Lancer DAST'
Frontend --> ms-execution : POST /api/security/scan/dast?projectId=1&environmentId;=1
ms-execution --> DB : SELECT environment (baseUrlApi)
ms-execution --> DB : INSERT security_scans (status=RUNNING, targetUrl=...)
ms-execution --> Frontend : 202 Accepted
ms-execution --> Docker : docker run zaproxy/zap-stable zap-baseline.py -t
Docker/ZAP --> Target App : HTTP Spider + Passive Scan + Active Scan
Target App --> Docker/ZAP : Responses
Docker/ZAP --> ms-exec : JSON report (site.alerts[])
ms-execution --> Parse : alerts[] --> SecurityVulnerability[]
ms-execution --> DB : INSERT security_vulnerabilities
ms-execution --> DB : UPDATE security_scans (status=COMPLETED)
```

11. Classification des Vulnerabilites

11.1 Mapping CWE vers OWASP Top 10 (2021)

CWE	Vulnerabilite	OWASP Top 10
CWE-89	SQL Injection	A03 - Injection
CWE-79	Cross-Site Scripting (XSS)	A03 - Injection
CWE-78	OS Command Injection	A03 - Injection
CWE-798	Hardcoded Credentials	A07 - Authentication Failures
CWE-327	Broken Cryptography	A02 - Cryptographic Failures
CWE-22	Path Traversal	A01 - Broken Access Control
CWE-502	Unsafe Deserialization	A08 - Software Integrity Failures
CWE-918	SSRF	A10 - SSRF
CWE-352	CSRF	A05 - Security Misconfiguration
CWE-611	XXE	A05 - Security Misconfiguration

11.2 Mapping des severites

Semgrep vers modele interne :

Semgrep Severity	Severite Interne
ERROR	HIGH
WARNING	MEDIUM
INFO	LOW

ZAP Risk Code vers modele interne :

ZAP Risk Code	Severite Interne
3 (High)	HIGH
2 (Medium)	MEDIUM
1 (Low)	LOW
0 (Informational)	INFO

CVSS Score vers modele interne (SCA) :

Score CVSS	Severite Interne
9.0 - 10.0	CRITICAL
7.0 - 8.9	HIGH
4.0 - 6.9	MEDIUM
0.1 - 3.9	LOW

12. Conclusion

Le module Security Center transforme la plateforme d'un simple outil d'automatisation de tests en une solution complete de DevSecOps. L'integration de Semgrep (SAST), OWASP ZAP (DAST), et OWASP Dependency-Check (SCA) permet une couverture de securite a 360 degres : analyse du code source, des applications en production, et des dependances tierces.

Les resultats sont centralises dans PostgreSQL via des entites JPA partagees entre ms-execution (ecriture des resultats de scan) et ms_gestion (lecture et agregation pour le dashboard). Le frontend Next.js offre un dashboard interactif avec 9 onglets couvrant tous les aspects de la securite : vue d'ensemble, SAST, DAST, OWASP Top 10, gestion des vulnerabilites, conformite, tendances, pipeline CI/CD, et rapports.

L'architecture microservices permet une execution asynchrone des scans sans bloquer l'interface utilisateur, et le mecanisme de graceful fallback assure que le dashboard reste fonctionnel meme lorsque le backend est indisponible.

Couche	Technologie	Role
Frontend	Next.js 16 + Recharts + shadcn/ui	Dashboard interactif, declenchement des scans
API Gateway	Next.js API Routes	Proxy vers ms-execution et ms_gestion
Execution	Spring Boot (ms-execution)	Clone Git, lancement des outils, parsing, PDF
Gestion	Spring Boot (ms_gestion)	Dashboard KPIs, CRUD vulnerabilites, conformite
Stockage	PostgreSQL 15	Tables security_scans, security_vulnerabilities
SAST	Semgrep CLI	Analyse statique du code source
DAST	OWASP ZAP (Docker)	Analyse dynamique des applications web
SCA	OWASP Dependency-Check	Audit des dependances Maven
Rapport	iTextPDF 7	Generation de rapports PDF professionnels